

aelion

Sprint 2 · US2.1 · Séance 13

Tracking · Artefacts · Registry

MLOps léger :

tracking & artefacts — MLflow

Kick-off Sprint 2 — US2.1

Nous entrons dans le Sprint 2 — 5 User Stories à livrer

US2.1

MLOps : tracking & artefacts

→ Séance 13 (aujourd'hui)

US2.6

Explicabilité SHAP

→ Séance 14

US2.2

Dataset images vision

→ Séance 15

US2.3

Auto-encodeur CNN

→ Séances 16-17

US2.4

Éco-conception Optuna

→ Séances 18-19

Product Owner

« Le modèle de la semaine dernière avait un meilleur Recall. Tu peux retrouver les paramètres ? »

Developer

« Hm... c'est dans `model_v3_final_bis.ipynb`... ou peut-être v4... Attends je cherche dans mes notes... »

Solution : MLflow loggue automatiquement TOUT — params, métriques, artefacts, versions

01

Architecture MLflow



Composants · concepts · code de base

Tracking
Server

Enregistre : params,
métriques, tags, runs

```
log_param()  
log_metric()
```

Artifact
Store

Stocke les fichiers :
modèles, graphiques, CSV

```
log_artifact()  
log_model()
```

Model
Registry

Versionne les modèles
Staging → Production
→ Archived

```
register_model()  
load_model()
```

MLflow
UI

Interface web
Comparaison visuelle
des runs

```
mlflow ui
```

Experiment

Groupe logique de runs
Ex : 'InduSense_Panne_Detection'

```
mlflow.set_experiment('...')
```

Run

Une exécution d'entraînement
Isolée, horodatée, identifiée

```
mlflow.start_run()
```

Params

Hyperparamètres (fixes)
Enregistrés une fois par run

```
mlflow.log_param(k, v)
```

Metrics

Valeurs numériques
Peuvent varier (epoch, step)

```
mlflow.log_metric(k, v)
```

Artifacts

Fichiers produits
.pkl, .png, .csv

```
mlflow.log_artifact(path)
```

Tags

Métadonnées libres
auteur, dataset, branche

```
mlflow.set_tag(k, v)
```

Python · MLflow

```
import mlflow
import mlflow.sklearn
from sklearn.metrics import recall_score, f1_score

mlflow.set_experiment("InduSense_Panne_Detection")

with mlflow.start_run(run_name="RF_baseline_v1"):
    # 1. Log des paramètres (AVANT l'entraînement)
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 10)
    mlflow.log_param("class_weight", "balanced")
    mlflow.log_param("random_seed", 42)

    # 2. Entraînement
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)

    # 3. Log des métriques
    mlflow.log_metric("recall", recall_score(y_test, y_pred))
    mlflow.log_metric("f1", f1_score(y_test, y_pred))

    # 4. Log du modèle
    mlflow.sklearn.log_model(rf, "model")

    # 5. Tags
    mlflow.set_tag("author", "marine.lannes")
    mlflow.set_tag("dataset_version", "v1.0")

print("Run ID:", mlflow.last_active_run().info.run_id)
```

MLflow capture aussi automatiquement

- Run ID unique
- Timestamp start / end
- Durée d'exécution
- Git commit hash
- User

Lancer l'UI

mlflow ui

→ <http://localhost:5000>

aelien

02

Capturer & comparer



[Autolog](#) · [search_runs](#) · [Model Registry](#)

Autolog — 1 ligne pour tout capturer

aelion.fr

Python · MLflow autolog

```
# Pour sklearn (Random Forest, Logistic Regression...)
mlflow.sklearn.autolog()
```

```
with mlflow.start_run(run_name="RF_autolog"):
    rf.fit(X_train, y_train)
    # → Capture automatiquement :
    #   params (n_estimators, max_depth, ...)
    #   metrics (accuracy, recall, ...)
    #   model sérialisé
    #   feature importances
    #   confusion matrix
```

```
# Pour XGBoost
mlflow.xgboost.autolog()
```

```
with mlflow.start_run(run_name="XGB_autolog"):
    xgb_model.fit(X_train, y_train,
                  eval_set=[(X_test, y_test)],
                  verbose=False)
    # → Capture loss curves, métriques, params
```

Avantages de l'autolog

- ✓ Zéro oubli — tout est capturé
- ✓ Courbes d'entraînement (XGB)
- ✓ Model signature automatique
- ✓ Feature importance (RF)
- ✓ Reproductibilité garantie

Limites : certains modèles non supportés. Combiner autolog + log_metric pour les métriques custom.

Comparer les runs — API Python

Python · MLflow.search_runs

```
import mlflow
import pandas as pd

# Récupérer tous les runs d'un experiment
runs = mlflow.search_runs(
    experiment_names=["InduSense_Panne_Detection"],
    order_by=["metrics.recall DESC"]
)

# Colonnes utiles
cols = ['run_id', 'params.n_estimators', 'params.max_depth',
        'metrics.recall', 'metrics.f1', 'tags.mlflow.runName']
print(runs[cols].head(10))

# Trouver le meilleur run
best_run = runs.loc[runs['metrics.recall'].idxmax()]
print(f"\nMeilleur run : {best_run['run_id'][:8]}")
print(f"  Recall : {best_run['metrics.recall']:.3f}")

# Charger le modèle du meilleur run
best_model = mlflow.sklearn.load_model(
    f"runs:{best_run['run_id']}/model"
)
```

Dans l'UI MLflow

Parallel Coordinates Plot

Visualiser l'impact de chaque param sur le Recall

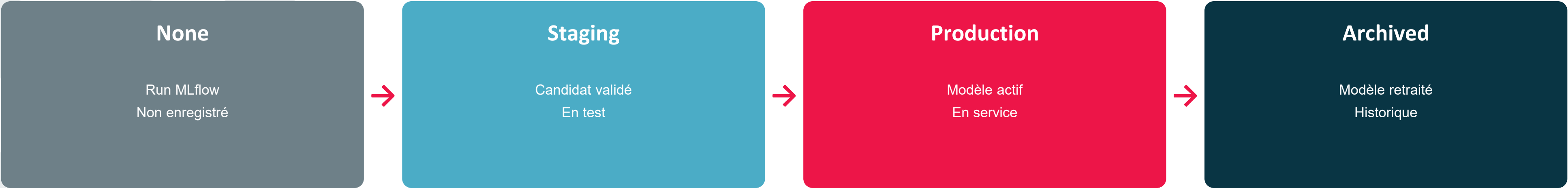
Scatter Plot

Comparer recall vs f1

Tableau tri / filtre

Trier par métrique

Model Registry — Cycle de vie des modèles



```
Python · MlflowClient

# Enregistrer dans le registry
with mlflow.start_run():
    mlflow.sklearn.log_model(
        rf, "model",
        registered_model_name="InduSense_PanneDetection"
    )

# Promouvoir en staging
client = mlflow.MlflowClient()
client.transition_model_version_stage(
    name="InduSense_PanneDetection",
    version=1, stage="Staging"
)

# Charger depuis le registry
model = mlflow.pyfunc.load_model("models:/InduSense_PanneDetection/Staging")
preds = model.predict(X_test)
```

03

Qualité & reproductibilité



Bonnes pratiques · tests pytest

Python · reproductibilité MLflow

```
# 1. Seeds partout
import numpy as np, random, os
SEED = 42
np.random.seed(SEED); random.seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)
mlflow.log_param("random_seed", SEED)

# 2. Hash du dataset
import hashlib, pandas as pd
def hash_df(df):
    return hashlib.md5(
        pd.util.hash_pandas_object(df).values
    ).hexdigest()[:8]
mlflow.set_tag("dataset_hash", hash_df(X_train))

# 3. Versions des librairies
import sklearn, xgboost
mlflow.set_tag("sklearn_version", sklearn.__version__)
mlflow.set_tag("xgboost_version", xgboost.__version__)

# 4. Naming convention
mlflow.set_experiment("InduSense_Panne_RF")
with mlflow.start_run(run_name="RF_v2_balanced_2026-05"):
    ...
```

Seeds fixes

`np.random.seed + random_state=42 + PYTHONHASHSEED`

Hash dataset

Détecter si les données ont changé entre les runs

Tags versions

`sklearn, xgboost, python` — pour reproduire en cas de bug

Naming clair

Experiment + Run nommés explicitement

Log avant train

Log les params AVANT de lancer l'entraînement

Tests pytest — Valider le modèle candidate

Un modèle en staging doit être testé programmatiquement avant d'aller en production.

tests/test_model_candidate.py

```
import mlflow, pytest, numpy as np
from sklearn.metrics import recall_score

MODEL_URI = "models:/InduSense_PanneDetection/Staging"

@pytest.fixture(scope="module")
def model():
    return mlflow.pyfunc.load_model(MODEL_URI)

def test_model_loads(model):
    assert model is not None

def test_model_predicts_single(model):
    # 1 sample : temp_moy, temp_max, pression, vibration, heures_fonc, age_machine
    X_sample = np.array([[65.0, 78.0, 4.2, 1.1, 2000, 5]])
    pred = model.predict(X_sample)
    assert len(pred) == 1
    assert pred[0] in [0, 1]

def test_model_predicts_batch(model):
    X_batch = np.random.rand(50, 6)
    preds = model.predict(X_batch)
    assert len(preds) == 50
    assert all(p in [0, 1] for p in preds)

def test_model_recall_above_threshold(model):
    """ Valider que le Recall minimum est atteint """
    recall = recall_score(y_test, model.predict(X_test))
    assert recall >= 0.70, f"Recall {recall:.3f} < seuil 0.70"

# Lancer : pytest tests/ -v
```

aelien

04

TP InduSense



Tracking · registry · tests

Objectif du TP

- Instrumenter le modèle validé en S12 avec MLflow
- Logger plusieurs runs (RF, XGBoost, LR)
- Comparer dans l'UI
- Publier le meilleur en staging dans le registry

Durée

2 h

Modèle
final S12

Livrables : Notebook MLflow · 3 runs distincts · Modèle staging · Tests pytest

01

Setup & premiers runs

🕒 50 min

- `mlflow.set_experiment('InduSense_Panne')`
- Run RF avec autolog
- Run XGBoost avec autolog
- Run manuel : `log_param` + `log_metric` + `log_artifact`
- Naviguer dans l'UI : `mlflow ui`

02

Comparaison & registry

🕒 40 min

- `mlflow.search_runs()` pour comparer
- Promouvoir le meilleur → Staging
- Charger depuis le registry
- Vérifier la prédiction

03

Tests & README

🕒 30 min

- `pytest test_model_candidate.py`
- Rédiger `README.md` du modèle
- Définir les métriques de monitoring

MLflow

Tracking & Registry

```
pip install mlflow
```

Docs : mlflow.org

mlflow.sklearn

Autolog sklearn

```
mlflow.sklearn.autolog()
```

log_model, load_model

mlflow.xgboost

Autolog XGBoost

```
mlflow.xgboost.autolog()
```

Capture loss curves

MlflowClient

API Python

```
search_runs()
```

```
transition_model_version_stage()
```

À utiliser dans Claude / ChatGPT

prompt.txt

"Aide-moi à instrumenter un pipeline ML avec MLflow.

Je veux :

- 1) Un run avec `mlflow.sklearn.autolog()` pour Random Forest
(`n_estimators=100`, `class_weight='balanced'`, `random_state=42`)
- 2) Un run avec `mlflow.xgboost.autolog()` pour XGBoost
(`learning_rate=0.1`, `scale_pos_weight=40`)
- 3) Un run manuel avec `log_param` / `log_metric` pour Logistic Reg.
+ sauvegarder la confusion matrix comme artefact

Ensuite :

- 4) Comparer avec `mlflow.search_runs()` et afficher le meilleur Recall
- 5) Enregistrer le meilleur dans le Model Registry sous
'InduSense_PanneDetection' en `stage='Staging'`
- 6) Charger depuis le registry et faire une prédiction test

`mlflow.set_experiment('InduSense_Panne')` est déjà configuré.

`X_train`, `X_test`, `y_train`, `y_test` sont disponibles."

5 points clés à retenir

aelion.fr

1

Un run = traçabilité complète

Params + métriques + artefacts + tags — tout versionné

2

Autolog = 1 ligne

`mlflow.sklearn.autolog()` capture tout sans effort

3

Model Registry

Cycle de vie formalisé : staging → production → archived

4

Seeds + hash dataset

Reproductibilité garantie entre les runs

5

Tests pytest

Valider le candidate avant de le promouvoir

aelion

À livrer en fin de séance

- ✓ Notebook avec au moins 3 runs MLflow distincts
- ✓ Modèle 'candidate' en Staging dans le registry
- ✓ Test pytest passant
- ✓ README du modèle candidate

Séance 14 — Explicabilité tabulaire (SHAP)

« *Modèle tracé et versionné — pourquoi décide-t-il ?* »

- SHAP values
- Story métier
- Rapport d'explicabilité

S11 → S12 → S13 (MLflow) → S14 (SHAP)



Merci de votre attention !

aelion

95 chemin de Gabardie 31200 Toulouse
05 61 49 42 60 - contact@aelion.fr

aelion.fr

